KeelNet: Controlled Abstention for Evidence-Grounded Question Answering

Naufal M. Kamaruddin Ideun Jeeong Author 3

March 26, 2026

Abstract

KeelNet studies a narrower and more operational version of hallucination control: evidence-grounded question answering with an explicit answer-or-ABSTAIN decision. The current implementation now reports two stages on SQuAD 2.0 with one question and one evidence passage as input. Stage 1 compares an answer-only baseline with an abstain-aware variant; the abstain-aware model reduces unsupported answers on the development split from 100.00% to 27.97% and improves decision-aware overall F_1 from 42.35 to 69.40, while answerable F_1 decreases from 84.82 to 66.76. Stage 2 adds a support verifier on top of the Stage 1 abstain-aware system. The verifier reaches 79.79 support F_1 on the development split at a selected support threshold of 0.25, but the resulting gated QA system changes end-to-end metrics only marginally: overall F_1 moves from 69.40 to 69.42, unsupported answer rate from 27.97% to 27.57%, and answerable F_1 from 66.76 to 66.41. These findings support the claim that explicit abstention supervision and support verification are both learnable, but they also show that the current support-gating policy is too permissive to deliver a large end-to-end gain. That result motivates the next stages on calibration, tighter unsupported-answer control, retrieval, and adaptive constraint balancing.

1 Introduction

Large language models can produce answers that are fluent and plausible while still being unsupported by the available evidence. For this project, the main concern is not hallucination as a vague catch-all label, but unsupported answering delivered as if it were reliable. That framing matters because improving average accuracy does not automatically reduce overconfident unsupported outputs.

This paper takes a deliberately narrow starting point. Instead of attempting to solve hallucination in general, we study a controlled evidence-grounded question answering task in which a system must either answer or abstain. The current implementation uses SQuAD 2.0 [2] with one question and one evidence passage as input. Stage 1 compares two training regimes:

1. a baseline trained only on answerable examples, and

2. an abstain-aware variant trained to treat unsupported cases as a first-class target behavior.

Stage 2 then adds a verifier over question, passage, and predicted answer in order to estimate whether an answer is actually supported before it is shown to the user.

The central hypothesis is simple: explicit abstention supervision should reduce unsupported answers without causing an unacceptable collapse in answer quality on answerable questions. That hypothesis is narrow enough to evaluate cleanly, but still meaningful as a first step toward a broader program on grounded answering, support verification, and calibrated uncertainty. The next immediate question is whether a learned support signal can recover some of the

remaining quality-versus-grounding gap without undoing the abstention gain from Stage 1.

The intended application setting is also narrow. KeelNet is most naturally suited to bounded, evidence-grounded QA scenarios in which answers should be tied to supplied documents and abstention is preferable to unsupported guessing, such as internal knowledge assistants, support systems over approved documentation, and policy or compliance QA. It is not intended as a general solution for open-domain truthfulness, balanced opinion generation, or fairness beyond the scope of the provided evidence.

This framing is also consistent with recent work arguing that hallucination persists partly because modern evaluation pipelines reward guessing more than appropriate uncertainty [1]. Our contribution is narrower and more operational: rather than explaining hallucination in general, Stage 1 tests whether an evidence-grounded QA system can be trained and evaluated so that abstention is treated as a useful target behavior instead of a failure by default, and Stage 2 tests whether a learned support verifier can improve downstream answer-versus-abstain decisions on top of that base system.

This LaTeX scaffold is intentionally written as a living project paper. Some sections already reflect stable design decisions, while others are structured to absorb final metrics, figures, and ablations once the experimental results are locked in.

2 Problem Setup

The current problem setting is evidence-grounded extractive question answering with a constrained binary action:

$$f(q, e) \rightarrow \{\text{answer span}, \textsc{abstain}\}, \tag{1}$$

where q is a question and e is a single evidence passage.

The Stage 1 scope is intentionally strict:

- Dataset: SQuAD 2.0 only.
- Input: one question and one evidence passage.
- Output: extractive answer span or ABSTAIN.
- Current implementation goal: reduce unsupported answers without sharply degrading performance on answerable questions.

This scope excludes retrieval, multi-hop reasoning, custom architectures, long-form generation, and broader claim-level verification. Those extensions are part of later roadmap stages, but they are out of scope for the current paper draft's core experimental claim.

We evaluate the task with metrics that separate utility from caution. For answerable examples, we track exact match and token-level F1. For unanswerable examples, we track unsupported answer rate and abstention quality. This makes it possible to study the trade-off between being useful and being appropriately cautious instead of collapsing everything into a single accuracy number.

3 Method

Stage 1 uses a standard extractive question answering architecture with a binary external decision: the system must either return an answer span from the passage or output ABSTAIN. The mechanism is intentionally simple. Rather than introducing a separate verifier or controller at this stage, the project asks whether abstention can already emerge from standard extractive QA supervision when unanswerable cases are represented correctly during training.

3.1 Encoder and QA heads

Let the input be a question q and a passage p. After tokenization, the model receives a single packed sequence of the form

$$x = [\text{CLS}], q, [\text{SEP}], p, [\text{SEP}]. \tag{2}$$

The encoder produces contextual token representations

$$h_1, h_2, \ldots, h_T = \text{Enc}_\theta(x), \tag{3}$$

where T is the tokenized sequence length. In the default setup, the encoder is initialized from the DistilBERT checkpoint `di`

`stilbert-base-uncased`. The implementation goes through the Hugging Face `AutoModelForQuestionAnswering` interface so the backbone can be swapped later.

The question answering head is the standard two-logit extractive head. For each token representation h_t, the model predicts:

$$s_t = w_s^\top h_t + b_s, \tag{4}$$

$$e_t = w_e^\top h_t + b_e, \tag{5}$$

where s_t is the start logit and e_t is the end logit for token t.

For a candidate span (i, j) with $i \leq j$, the raw span score is

$$S(i, j) = s_i + e_j. \tag{6}$$

This is the core deep learning mechanism of Stage 1: the encoder learns a contextual representation of the question-passage pair, and the two linear heads learn where the answer should begin and end.

3.2 Training targets

The project compares two training modes under the same architecture, preprocessing, and optimization settings.

Baseline mode. The baseline is trained only on answerable examples. This makes it a standard extractive QA model: every training case has a supervised span target inside the passage.

Abstain-aware mode. The abstain-aware model is trained on both answerable and unanswerable examples. For answerable examples, the targets are the gold start and end token positions, just as in the baseline. For unanswerable examples, both the start and end targets are mapped to the CLS position.

This matters because it turns abstention into a first-class supervised outcome without changing the external architecture. The model is not given a separate abstention head. Instead, it learns to use the score of the CLS token as a null or no-answer signal.

The optimization objective is the standard averaged cross-entropy loss over start and end targets:

$$\mathcal{L} = \frac{1}{2}\Big(\mathrm{CE}(s, y^{\mathrm{start}}) + \mathrm{CE}(e, y^{\mathrm{end}})\Big), \tag{7}$$

where y^{start} and y^{end} are the gold token indices. For unanswerable examples in the abstain-aware setting,

$$y^{\mathrm{start}} = y^{\mathrm{end}} = c, \tag{8}$$

with c denoting the CLS index.

3.3 Long-context handling

Because passages can exceed the maximum encoder length, Stage 1 uses the standard sliding-window extractive QA preprocessing strategy. Each example may be expanded into multiple overlapping features with a fixed maximum sequence length and document stride. During training, each feature receives span supervision relative to its own window. During evaluation, the pipeline keeps the metadata needed to map feature-level predictions back to the original example.

This implementation detail is important in deep learning terms because the model is trained over token windows rather than over raw passages directly. The final example-level prediction is therefore an aggregation over feature-level start and end logits.

3.4 Inference mechanism

At inference time, the model produces start and end logits for every feature of an example. Stage 1 then computes two competing quantities:

1. the best valid answer span score across all candidate spans and all windows, and
2. the best null score, defined as the sum of the start and end logits at the CLS position.

If the best span is $(i^\star, j^\star)$, then the best answer score is

$$S^\star_{\mathrm{span}} = \max_{i,j \in \mathcal{V}} S(i, j), \tag{9}$$

where $\mathcal{V}$ contains only valid spans that satisfy the ordering and maximum-answer-length constraints.

The null score is

$$S_{\varnothing} = s_c + e_c, \tag{10}$$

again using the CLS position c.

The baseline model always outputs the highest-scoring span. The abstain-aware model compares the null score against the best span through a margin:

$$\Delta = S_{\varnothing} - S^{\star}_{\text{span}}. \tag{11}$$

The final decision rule is:

$$\hat{y} = \begin{cases} \text{ABSTAIN}, & \text{if } \Delta > \tau, \\ \text{best span}, & \text{otherwise}, \end{cases} \tag{12}$$

where τ is a validation-selected threshold.

Conceptually, this means Stage 1 does not learn a symbolic rule for when to refuse. It learns a representation in which the no-answer state competes directly against answer spans in the same scoring space.

3.5 Threshold selection

The abstention threshold is not fixed by hand in the default evaluation path. Instead, the code performs a simple search over a bounded range on the validation split. For each candidate threshold, the system post-processes predictions and measures Stage 1 metrics. The current implementation selects the threshold by prioritizing abstain F_1, with answerable F_1 used as a secondary tie-breaker.

This choice reflects the Stage 1 research question. The project is not only asking whether the model can refuse more often; it is asking whether it can reduce unsupported answers while preserving usefulness on answerable examples.

3.6 What Stage 1 proves

From a deep learning perspective, Stage 1 is a representation and decision control test. It asks whether a standard extractive QA network, with no additional verifier and no retrieval pipeline, can be pushed toward better answer-versus-abstain behavior purely through:

- different supervision over answerable versus unanswerable cases,
- a shared encoder and span head,
- the CLS-based null mechanism, and
- thresholded post-processing over the learned logit space.

If this works, then later stages can build on a meaningful base mechanism instead of introducing extra components before the basic abstention behavior is understood.

3.7 Stage 2 verifier and support gate

Stage 2 keeps the Stage 1 abstain-aware QA model fixed and adds a second model that judges whether a proposed answer is actually supported by the passage. This makes the overall system modular rather than jointly end-to-end: Stage 1 proposes an answer-or-ABSTAIN decision, and Stage 2 decides whether a predicted answer should be trusted.

Verification examples. The verifier is trained as a binary classification problem built from the same underlying SQuAD 2.0 splits. For each answerable training example, the pipeline creates one positive verification record using a gold answer and one or more negative records using sampled spans from the same passage that do not overlap strongly with the gold answer. For each unanswerable example, the pipeline creates only negative records by sampling unsupported spans from the passage. The result is a support-label dataset over tuples (q, p, a), where q is the question, p is the passage, and a is a candidate answer.

Verifier input and model. The verifier formats the first sequence as a short hypothesis-like query,

$$x^{(1)} = \texttt{Question:}\quad q\ \texttt{\textbackslash n Answer:}\quad a, \tag{13}$$

and uses the passage as the second sequence,

$$x^{(2)} = p. \tag{14}$$

These paired sequences are fed into a standard transformer sequence classifier initialized from the same `distilbert-base-uncased` checkpoint family as Stage 1, but with a two-class support head. If h_{CLS} is the final pooled representation, the verifier computes support logits

$$z = Wh_{\text{CLS}} + b, \tag{15}$$

followed by a softmax that estimates

$$P_\phi(\text{supported} \mid q, p, a). \tag{16}$$

Training uses standard cross-entropy over the binary support labels.

Inference and gating. At evaluation time, only examples that Stage 1 already labeled as `answer` are sent to the verifier. If the Stage 1 system abstains, the final decision remains ABSTAIN. If Stage 1 answers, Stage 2 attaches a support score $s_{\text{sup}} \in [0, 1]$ and applies a second threshold:

$$\hat{y}_{\text{final}} = \begin{cases} \text{answer}, & \text{if } \hat{y}_{\text{qa}} = \text{answer and } s_{\text{sup}} \geq \eta, \\ \text{ABSTAIN}, & \text{otherwise.} \end{cases} \tag{17}$$

Here η is a validation-selected support threshold. This means the Stage 2 system is a cascaded decision process: the QA model first decides whether answering is plausible at all, and the verifier then decides whether that proposed answer appears passage-supported.

Threshold search and Stage 2 objective. The support threshold is selected on the validation split by prioritizing support F_1, then gated overall F_1, and then lower unsupported-answer rate. This reflects the Stage 2 question directly: not only whether support is learnable, but whether a learned support signal can improve the final answer-versus-ABSTAIN decision. In deep learning terms, Stage 2 tests whether a second learned representation space over (q, p, a) can act as a useful post-hoc controller on top of the Stage 1 answer space.

4 Experimental Protocol

The active experiments are run from the project notebooks and Python modules in the repository. The current implementation uses a compact extractive QA setup with a default DistilBERT backbone and SQuAD 2.0 preprocessing.

Dataset split. The training pipeline creates deterministic train, validation, and development splits from SQuAD 2.0. For the answer-only baseline, the training set is restricted to answerable examples. For the abstain-aware setting, both answerable and unanswerable cases are kept. For Stage 2, the verifier is trained on examples derived from the Stage 1 QA outputs so that each instance contains a question, a passage, a predicted answer, and a support label.

Evaluation. Model outputs are post-processed into final decisions and scored with:

- answerable exact match,
- answerable F1,
- decision-aware overall exact match,
- decision-aware overall F1,
- unsupported answer rate, and
- abstain precision, recall, and F1.

The decision-aware overall metrics give full credit to correct abstentions on unanswerable examples, so they do not penalize uncertainty by construction. For Stage 2, we additionally report support accuracy, precision, recall, and F1, together with supported-answer rate and contradiction rate for the verifier and its downstream gated QA behavior.

Execution workflow. The current team workflow runs experiments from browser Google Colab while editing source code locally in VS Code. The notebook supports both hosted Colab and Colab connected to a local runtime; the Stage 1 and Stage 2 runs reported here used a

local runtime backed by the project's WSL environment and GPU. Artifacts are stored under the project runtime directory, and code changes are synchronized through Git branches.

What to lock before the final paper. Before turning this draft into a submission-ready paper, we should freeze:

- the final model checkpoint choice,
- the exact QA and verifier threshold-selection policies,
- the final metric table,
- at least one qualitative error analysis slice, and
- the figure or table that best shows the utility-versus-abstention trade-off.

5 Results

Table 1 is the core Stage 1 comparison. It reports one stable baseline run and one stable abstain-aware run under matched training settings.

The baseline is trained only on answerable examples, while the abstain-aware variant includes explicit supervision for unanswerable cases. For Stage 1, a win is defined conservatively: unsupported-answer rate should decrease, answerable performance should remain usable, and abstain quality should improve without the model collapsing into trivial refusal behavior. Under that conservative criterion, the current Stage 1 result is a qualified success. The abstain-aware model cuts unsupported-answer rate by 72.03 points, from 100.00% to 27.97%, and raises decision-aware overall F_1 by 27.05 points, from 42.35 to 69.40. At the same time, answerable F_1 drops by 18.07 points, from 84.82 to 66.76, so the improvement comes with a real answer-quality cost rather than a free gain.

Because the abstain-aware system uses a validation-selected threshold, the main table should be read together with a threshold sweep rather than as a single isolated operating point.

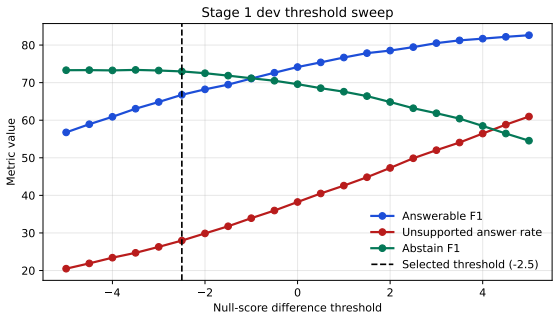

Figure 1: Threshold sweep for the abstain-aware model on the development split. The vertical line marks the validation-selected threshold of -2.5.

Threshold interpretation. Figure 1 shows the expected trade-off as the threshold moves toward less negative values. Across the dev sweep from -3.5 to -1.5, answerable F_1 rises from 63.06 to 69.47, but unsupported-answer rate also rises from 24.71% to 31.77%. Abstain F_1 changes more gently, moving from 73.39 to 71.86 across the same range. The selected threshold of -2.5 sits in the middle of this trade-off region: neighboring points at -3.0 and -2.0 have similar dev overall F_1 (69.29 and 69.17 versus 69.40 at -2.5), which suggests that the gain is not coming from a single brittle threshold setting.

Stage 1 takeaway. The main question for Stage 1 was whether explicit abstention supervision could correct the baseline's pathological always-answer behavior. On that question, the answer is yes. The baseline answers every unsupported example, while the abstain-aware model learns a meaningful abstain policy and achieves 72.98 abstain F_1 on the dev split. The cost is that answer extraction quality remains substantially worse than the baseline on answerable questions, which gives a clear target for Stage 2 and later stages: preserve the abstention gain while recovering answer quality.

Stage 2 support-verification result. Stage 2 keeps the Stage 1 abstain-aware QA model fixed and adds a verifier over question, passage, and predicted answer. The selected support threshold is 0.25. On the validation split, the

Table 1: Main Stage 1 comparison between the answer-only baseline and the abstain-aware model on the held-out development split.

Model	Answerable F1	Decision-aware Overall F1	Unsupported Rate	Abstain F1
Baseline	84.82	42.35	100.00	–
Abstain-aware	66.76	69.40	27.97	72.98

Table 2: Stage 2 support verification on top of the Stage 1 abstain-aware QA system, evaluated on the held-out development split at support threshold 0.25. All values are percentages.

	Verifier			End-to-end QA		
System	Prec.	Rec.	F1	Ans. F1	Overall F1	Unsup. Rate
Stage 1 abstain QA	–	–	–	66.76	69.40	27.97
Stage 2 gated QA	66.62	99.43	79.79	66.41	69.42	27.57
Delta vs. Stage 1	–	–	–	-0.35	+0.03	-0.40

verifier reaches 87.71 support F_1. On the held-out development split, it reaches 79.79 support F_1, with a 66.26% supported-answer rate and a 33.00% contradiction rate. When that support signal is used to gate the Stage 1 abstain-aware system, the end-to-end QA metrics move only slightly: decision-aware overall F_1 rises from 69.40 to 69.42 and unsupported-answer rate falls from 27.97% to 27.57%, while answerable F_1 slips from 66.76 to 66.41.

How to read the Stage 2 outcome. The positive result is that the verifier does learn a meaningful support signal instead of collapsing completely. The limiting result is that this signal does not yet translate into a large downstream improvement. On the dev split, verifier recall is extremely high at 99.43%, but precision is only 66.62%, and the model predicts support for 98.88% of answered cases. That makes the current gating policy too permissive: the verifier mostly confirms answers rather than filtering unsupported ones. As a result, Stage 2 should be read as a systems success but not yet as a decisive end-to-end win. It narrows the next bottleneck clearly: later stages need better calibration and control over when support scores are allowed to suppress or permit answers.

Representative failure cases. The saved dev predictions make the current bottleneck concrete. On the unanswerable question *"What is France a region of?"*, the Stage 1 baseline answers *"Normandy"*, the Stage 1 abstain-aware model still answers *"Normandy"*, and Stage 2 assigns that answer a support score of 0.9998 and keeps the final decision as `answer`. This is a clean false-positive support error: the passage contains both *France* and *Normandy*, but the relation in the question is reversed relative to the evidence. By contrast, on the answerable question *"If input size is equal to n, what can respectively be assumed is the function of n?"*, the Stage 1 abstain-aware model correctly predicts *"the time taken"*, but Stage 2 assigns that answer a support score of only 0.0052 and gates it to ABSTAIN. This is the complementary false-negative support error: the verifier can over-penalize short abstract answers even when the QA model is already correct. Taken together, these examples match the aggregate metrics: the current verifier is strong enough to learn support as a signal, but not yet selective enough to improve the final decision reliably.

6 Discussion and Limitations

The current project deliberately favors scope control over breadth. That is a strength for the first paper draft, but it also creates limitations that should be stated clearly.

Current limitations.

- The active implementation currently covers Stage 1 and Stage 2 only.
- The task uses a single provided evidence passage rather than retrieved evidence.

- The action space is intentionally narrow and does not yet cover richer support labels or multi-hop settings.
- The current results will reflect one controlled dataset setup rather than a broad claim about hallucination in general.

Intended deployment scope. The current approach is best interpreted as a method for high-trust, document-bounded QA, not as a general-purpose opinion or advice generator. Its most plausible deployment settings are ones in which the evidence source is explicit, the answer should remain short and verifiable, and abstaining is preferable to guessing. If the provided evidence is incomplete, outdated, or systematically biased, the method may still produce a well-grounded but incomplete answer, so groundedness should not be confused with neutrality or full real-world balance.

How to interpret the current result. The current evidence supports two related but different claims. First, the Stage 1 run shows that explicit abstention supervision and abstention-aware evaluation can strongly reduce unsupported answers in a simple extractive QA system. Second, the Stage 2 run shows that support verification is learnable on top of that abstain-aware base. However, the current verifier mostly predicts support rather than intervening aggressively enough to shift end-to-end QA behavior. The result should therefore be read as a trade-off improvement plus a partially successful extension, not yet as a generally better grounded QA model.

Why this stage is still useful. Even under its narrow scope, the first two stages now test two defensible claims with clean supervision and interpretable failure modes. Stage 1 shows that abstention can be taught. Stage 2 shows that support can be predicted, but also reveals the remaining bottleneck clearly: the system still needs better calibration and tighter decision control before a support signal becomes a strong downstream filter.

Legacy plan and revision. The original roadmap after Stage 1 was to move directly from Stage 2 support verification into Stage 4 unsupported-confidence control, while treating Stage 3 calibration as supporting work and deferring retrieval to a later realism check. That legacy plan assumed that once support became learnable, the raw verifier scores would already be good enough to drive downstream control. Stage 2 changed that assumption. On the dev split, support recall reaches 99.43%, but support precision is only 66.62%, the verifier predicts support for 98.88% of answered cases, and the gated system improves overall F_1 by only 0.03 points. So the long-term goal stays the same, but the immediate plan changes: calibration now becomes the next dependency rather than optional supporting work, because the current support scores are still too permissive for a strong fixed controller.

Next paper-facing extensions. With Stage 1 and Stage 2 now in place, the most natural paper extensions are:

- calibrating confidence against correctness and support,
- making the verifier less permissive on unsupported answers,
- testing retrieval noise, and
- comparing adaptive balancing against simpler fixed baselines.

7 Conclusion

This draft paper frames KeelNet as a staged research program for evidence-grounded question answering with controlled abstention. The current implemented contribution now covers Stage 1 and Stage 2 under a controlled SQuAD 2.0 setting: first an answer-only baseline versus an abstain-aware alternative, then a learned support verifier on top of the abstain-aware system.

The current evidence is encouraging but mixed in the right way for a staged project.

Stage 1 substantially improves unsupported-answer behavior and decision-aware overall performance, but it still gives up too much answerable-question quality. Stage 2 then shows that support verification is learnable, yet the present gating policy is too permissive to create a large end-to-end gain. That means the project has cleared two useful bars: abstention supervision works, and support prediction is viable. At the same time, the results sharply motivate the next stages, especially calibration and tighter unsupported-answer control, because they now look like the missing pieces needed to turn a learned support signal into a stronger downstream decision policy.

References

[1] Adam Tauman Kalai, Ofir Nachum, Santosh S. Vempala, and Edwin Zhang. Why language models hallucinate, 2025. URL https://arxiv.org/abs/2509.04664.

[2] Pranav Rajpurkar, Robin Jia, and Percy Liang. Know what you don't know: Unanswerable questions for SQuAD. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 784–789, 2018.